

2E Phase 5 Writeup

Common subexpression elimination (CSE)

The first optimization we considered was common subexpression elimination (CSE). Our motivation for implementing CSE was that reducing the number of repeated computations by eliminating common subexpressions would improve performance significantly. Our CSE optimization keeps track of multiple mappings to start. These mappings are a map from variables to their values, expressions to their values, and expressions to the temporary variables created for them. Then, the optimization looks at each instruction and performs common subexpression elimination depending on the type of instruction. For binary operations, the variable and expression values are retrieved from our maps or initialized if not found. Since some binary operations are commutative, both orders of the expression are checked. Finally, the temporary that goes with this expression is saved, and the instruction is replaced with a version that has common subexpressions eliminated. Unary operations are handled in a similar way, except that there is only one ordering of the terms in the expression, so multiple orders do not need to be checked. If the optimization comes across a call instruction, any expression that has a global variable has to be removed from our mappings, since functions may modify the value of global variables. If the optimization comes across a copy instruction, the value of the variable in our mappings has to be updated to account for the new value. If the optimization comes across an array check instruction, it is replaced by an empty instruction if the array element value has been seen before.

The following is a short Decaf program which demonstrates our CSE optimization in action:

```
void main() {  
    int a, b;  
    a = 1 + 2;  
    b = 2 + 1;  
}
```

The full assembly code can be found under our repository's docs folder, in the phase5-ex directory, under the filenames "cse-before.s" and "cse-after.s". The assembly code is identical, other than the following diff:

```
<    # b: stack[-16] = 2 + 1  
<    movq $2, %rax
```

```

<    addq $1, %rax
---
>    # b: stack[-16] = a: stack[-8]
>    movq -8(%rbp), %rax

```

Lines from before the optimization are marked with a left arrow, and lines from after the optimization are marked with a right arrow. Before the optimization, our compiler did not recognize that the values computed by a and b are the same, so it unnecessarily recomputes the value for $2 + 1$ when it was already computed for a. After the optimization, our compiler recognizes that the value computed for a, $1 + 2$, is the same value computed for b, $2 + 1$. As a result, instead of moving 2 then adding 1, it saves an instruction and calculation by moving the value for a instead.

Our CSE optimization was benchmarked against the derby test, and compared against no optimizations. Without any optimizations, the average run time of the derby test on Athena is 804 ms. With only CSE enabled, the average run time of the derby test decreases to 730 ms.

Copy propagation (CP)

The second optimization we considered was implementing copy propagation (CP). Our motivation for implementing CP was that having constants and copies propagate would reduce unnecessary computation and memory access, resulting in better performance. For this optimization, we were able to implement a lattice abstraction and a fixed-point worklist algorithm so that one pass of CP will allow copies to propagate throughout the program. Our CP optimization keeps track of a mapping of a variable to the original variable to which it was assigned. Our transfer function works at the instruction level, where special care has to be taken with function calls due to globals potentially being modified, and copy instructions which can cause an infinite loop if not handled correctly. Finally, we initialize our lattice bottom with all copy instructions with a different source and destination that operate on non-array values and constants, and we initialize our in and out bit vectors based on our transfer function. A standard fixed point worklist algorithm can be used to perform CP on each basic block of the program.

The following is a small Decaf program which demonstrates our CP optimization in action:

```

void main ( ) {
    int a, b;
    a = 0;
}

```

```
b = a;  
}
```

The full assembly code can be found under our repository's docs folder, in the phase5-ex directory, under the filenames "cp-before.s" and "cp-after.s". The assembly code is identical, other than the following diff:

```
<    # b: stack[-16] = a: stack[-8]  
<    movq -8(%rbp), %rax  
---  
>    # b: stack[-16] = 0  
>    movq $0, %rax
```

Lines from before the optimization are marked with a left arrow, and lines from after the optimization are marked with a right arrow. Before the optimization, our compiler did not recognize that the value assigned to a is also assigned to b. As a result, our compiler generates code that unnecessarily accesses the value for a to copy it to b. After the optimization, our compiler recognizes that the value for a was not modified before being assigned to b, so the value for a gets propagated down and the value for b is assigned directly. As a result, our compiler saves multiple memory accesses due to our CP optimization.

Our CP optimization was benchmarked against the derby test, and compared against no optimization and with CSE enabled as well. Without any optimizations, the average run time of the derby test on Athena is 804 ms. With only CP enabled, the average run time of the derby test decreases to 793 ms, most likely due to many benefits of CP not realized until after DCE. With CSE enabled, the average run time of the derby test decreases to 708 ms, with the majority of the time savings being from CSE, and some from propagation of found common subexpressions.

Dead code elimination (DCE)

The third optimization we considered was implementing dead code elimination (DCE). Our motivation for implementing DCE was that removing code that computes values that do not get used reduces unnecessary computation and memory accesses, and should give an improvement to performance. We were able to implement this optimization using our lattice abstraction that was created for our copy propagation optimization, allowing us to use a fixed-point worklist algorithm so that one pass of DCE will remove all dead code from a program. The bottom of our lattice is initialized to be an empty set, since our optimization is optimistic and assumes nothing is used. Our least

upper bound is defined as the union of the set of variables used between two sets, since we want to keep any code that is used later in the program. Our DCE decides whether to keep an instruction or not based on the type of instruction it is. Since function calls may modify globals or be callouts, we must keep those. Otherwise, if the destination or arguments of an instruction are not used later in the program, the instruction is eliminated. Our transfer function at the instruction level generates a bit vector of the values an instruction uses at or after that point in the program, and at the block level generates a bit vector of all values used within a block. With our least upper bound, transfer function, and lattice initialized, we are then able to use a standard worklist fixed-point algorithm to remove dead code throughout any program.

The following is a small Decaf program which demonstrates our DCE optimization in action:

```
int get_int (int x) {
    return x;
}

void main ( ) {
    int a, b;
    a = 0;
    b = 1;
    get_int(b);
}
```

The full assembly code can be found under our repository's docs folder, in the phase5-ex directory, under the filenames "dce-before.s" and "dce-after.s". The assembly code is identical, other than the following diff:

```
29c29
<     addq $-32, %rsp
---
>     addq $-16, %rsp
31c31
<     # init a: stack[-8]
---
>     # init b: stack[-8]
33,38c33
<     # init b: stack[-16]
<     movq $0, -16(%rbp)
```

```

<    # a: stack[-8] = 0
<    movq $0, %rax
<    movq %rax, -8(%rbp)
<    # b: stack[-16] = 1
---
>    # b: stack[-8] = 1
40,41c35,36
<    movq %rax, -16(%rbp)
<    # t3: stack[-24] = call get_int(b: stack[-16])
---
>    movq %rax, -8(%rbp)
>    # t3: stack[-16] = call get_int(b: stack[-8])
49c44
<    movq -16(%rbp), %rdi
---
>    movq -8(%rbp), %rdi
51c46
<    movq %rax, -24(%rbp)
---
>    movq %rax, -16(%rbp)

```

Lines from before the optimization are marked with a left arrow, and lines from after the optimization are marked with a right arrow. Before the optimization, our compiler did not recognize that the variable `a` is never used, so the line “`a = 0`” is dead code. As a result, the variable `a` gets allocated to the stack at an offset of `-8`, and its initial value of `0` gets copied to the stack. After our optimization, our compiler was able to recognize that the line “`a = 0`” is dead code. As a result, the compiler knows that `a` does not need to be allocated to the stack. We know this is the case because `b` gets allocated onto the stack at an offset of `-8`, when before `a` was allocated there and `b` was allocated at an offset of `-16`. This means that any instruction that involves `a` can be eliminated, reducing unnecessary instructions and memory accesses while giving a good performance improvement.

Our DCE optimization was benchmarked against the derby test, and compared against no optimizations, and with CSE and CP enabled as well. Without any optimizations, the average run time of the derby test on Athena is 804 ms. With only DCE enabled, the derby run time decreases to 793 ms. While this is within the margin of error, the assembly output is about 200 lines shorter, so DCE is making an impact on the instruction count. When combined with CSE and CP, the derby program run time

decreases to 686 ms. This means that when our dataflow optimizations are combined, programs should run about 18% faster than without any optimization.

Algebraic simplification

The fourth optimization we considered was implementing algebraic simplification. Our motivation for this was that reducing unnecessary computation in arithmetic operations would result in a performance improvement for programs that perform many arithmetic and logical operations. For multiplications and divisions, strength reduction is performed if a value is multiplied or divided by a power of 2, and the instruction is replaced with a left shift or right shift accordingly. For equality checking, the value of the equality is determined, and the instruction is replaced with a copy instruction that assigns the value of the equality to the original destination. The same is done for any comparison operations as well. For boolean logic, if both arguments of the operation are a boolean constant, then the instruction is replaced with a copy instruction assigning the result of the boolean operation to the original destination. Otherwise, if only one is a boolean constant, then the instruction gets replaced with a copy instruction assigning the boolean value of the operation depending on the value of the constant and type of operation. For *or* operations, the value assigned is true if either side of the *or* operation is true, or the value of the non-constant argument of either side of the *or* operation is false. For *and* operations, the value assigned is false if either side of the *and* operation is false, or the value of the non-constant argument if either side of the *and* operation is true. For integer arithmetic, if both arguments of the operation are integer constants, then the operation is replaced with a copy instruction assigning the value of the operation to the original destination. If only one argument of the operation is an integer constant, then some simplification can be done. If either argument for an add instruction is a 0, then the instruction is replaced with a copy instruction assigning the value of the other argument to the original destination. If either argument for a subtract instruction is a 0, then the instruction is replaced with a copy instruction assigning the positive value of the variable argument to the original destination if it is in the form of “ $x - 0$,” and the negative value of the variable argument to the original destination if it is in the form “ $0 - x$.” If either argument for a multiply instruction is a 1, then the instruction is replaced with a copy instruction assigning the value of the other argument to the original destination. If the divisor of a divide operation is a 1, then the instruction is replaced with a copy instruction assigning the value of the dividend to the original destination. Finally, for unary operations like negation and not operations, if the argument is a constant then the instruction is replaced with a copy instruction assigning the negated or inverted value to the original destination accordingly.

The following is a small Decaf program that demonstrates our algebraic simplification in action:

```

void main ( ) {
    int a, b;
    bool c;
    b = 1;
    a = 1 * 3;
    a = b * 16;
    c = true && false;
    a = b / 1;
    a = 1 * b;
    a = -(-b);
    a = b + 0;
    c = c || true;
    c = c && false;
    a = 0 - b;
}

```

The full assembly code can be found under our repository's docs folder, in the phase5-ex directory, under the filenames "algsimp-before.s" and "algsimp-after.s". The assembly code is identical, other than the following diff:

```

28,30c28,29
<    # a: stack[-8] = 1 * 3
<    movq $1, %rax
<    imulq $3, %rax
---
>    # a: stack[-8] = 3
>    movq $3, %rax
35c34
<    # a: stack[-8] = b: stack[-16] * 16
---
>    # a: stack[-8] = b: stack[-16] << 4
37c36
<    imulq $16, %rax
---
>    salq $4, %rax
42,44c41,42
<    # c: stack[-24] = true && false
<    movq $1, %rax

```

```

<    andq $0, %rax
---
>    # c: stack[-24] = false
>    movq $0, %rax
49c47
<    # a: stack[-8] = b: stack[-16] / 1
---
>    # a: stack[-8] = b: stack[-16]
51,53d48
<    cqo
<    movq $1, %rcx
<    idivq %rcx
58,60c53,54
<    # a: stack[-8] = 1 * b: stack[-16]
<    movq $1, %rax
<    imulq -16(%rbp), %rax
---
>    # a: stack[-8] = b: stack[-16]
>    movq -16(%rbp), %rax
68c62
<    # a: stack[-8] = b: stack[-16] + 0
---
>    # a: stack[-8] = b: stack[-16]
70d63
<    addq $0, %rax
75,77c68,69
<    # c: stack[-24] = c: stack[-24] || true
<    movq -24(%rbp), %rax
<    orq $1, %rax
---
>    # c: stack[-24] = true
>    movq $1, %rax
82,84c74,75
<    # c: stack[-24] = c: stack[-24] && false
<    movq -24(%rbp), %rax
<    andq $0, %rax
---
>    # c: stack[-24] = false
>    movq $0, %rax

```



```

89,91c80,82
<    # a: stack[-8] = 0 - b: stack[-16]
<    movq $0, %rax
<    subq -16(%rbp), %rax
---
>    # a: stack[-8] = - b: stack[-16]
>    movq -16(%rbp), %rax
>    neg  %rax

```

Lines from before the optimization are marked with a left arrow, and lines from after the optimization are marked with a right arrow. The diff at 28,30c28,29 shows that our compiler correctly simplifies multiplications where one argument is 1. The diffs at 35c34 and 37c36 show that our compiler correctly replaces multiplication by powers of 2 with a faster shift. The diff at 42,44c41,42 shows that our compiler evaluates constant argument operations and replaces the instruction with a copy instruction of the evaluated value instead. The diffs at 49c47 and 51,53d48 show that when a value is divided by 1, the instruction is replaced with a copy instruction with the dividend instead. The diff at 58,60c53,54 shows that our compiler correctly simplifies multiplying a variable by 1, and replaces it with a copy instruction with the variable instead. The diffs at 68c62 and 70d63 show that our compiler correctly simplifies adding 0 to a variable, and replaces it with a copy instruction with that variable instead. The diff at 75,77c68,69 shows that our compiler correctly simplifies an or operation between a variable and the constant true, and replaces it with a copy instruction with the value of true instead. The diff at 82,84c74,75 shows that our compiler correctly simplifies an *and* operation between a variable and the constant false, and replaces it with a copy instruction with the value of false instead. The diff at 82,84c74,75 shows that our compiler correctly simplifies subtracting a variable from 0, and replaces it with a faster negation instruction. Overall, these simplifications result in a lower number of instructions, replaces slow instructions with faster instructions, and avoids unnecessary memory accesses. As a result, our algebraic simplification optimization provides a good performance improvement.

Our algebraic optimization was benchmarked against the derby test, and compared against no optimization, and with all dataflow optimizations enabled as well. Without any optimizations, the average run time of the derby test on Athena is 804 ms. With only algebraic optimization enabled, the average run time of the derby test decreases to 802 ms. Our algebraic optimization only reduces the line count of the assembly output by about 30 lines, so a minimal difference in run time is expected. When combined with all dataflow optimizations, the average run time of the derby test is reduced to 660 ms. This means that when our dataflow optimizations are combined with

algebraic simplification, programs should run about 22% faster than without any optimization.

Register allocation

The final optimization we considered was implementing a graph coloring register allocator. Our motivation for implementing the register allocator is that accessing registers is much faster than accessing memory, so an optimal register allocator would provide major performance improvements to any program that does a lot of memory accesses. Our register allocator first builds definition and usage chains for each variable. Webs are then built from these chains using a union-find-like algorithm. After the webs are built, they can be used to create an interference graph. Webs are checked for interference by looping through the def-uses for both webs, and if a def or use for one web is found in between a def and use for another, the webs interfere. Once the graph is built, it can be colored. First, any webs that have a degree less than our register count get removed from the graph and are saved in a list. Then, any webs that have a degree greater than or equal to our register count get spilled, and do not receive a color. The heuristic for determining which web gets spilled is based on the maximal depth of the loop the variable is in, and the spill cost is 10 to the power of the loop depth. The web with the minimal spill cost then gets spilled. These two steps are repeated until the graph is empty. Then, the graph gets colored using a greedy coloring algorithm in the order that the webs were pruned. Coloring in this order ensures that the greedy coloring algorithm finds an optimal solution. After the graph has been colored, we then assign registers to variables based on the color the webs received.

The following is a small Decaf program that demonstrates our register allocator in action:

```
int func(int x) {
    return x;
}

void main() {
    int a, b, c, d, e, f;
    // a interferes with b, c, d, e
    // b interferes with a, e
    // c interferes with a, d, e
    // d interferes with a, c, e
    // e interferes with a, b, c, d, f
    // f interferes with e
}
```

```

a = 0;
e = 0;

c = 0;
d = 0;
func(c);
func(d);

b = 0;
func(a);
func(b);

f = 0;
func(e);
func(f);
}

```

The full assembly code can be found under our repository's docs folder, in the phase5-ex directory, under the filenames "regalloc-before.s" and "regalloc-after.s". The assembly code is identical, other than the following diff:

```

61c61
<    movq %rax, -8(%rbp)
---
>    movq %rax, %r15
64c64
<    movq %rax, -40(%rbp)
---
>    movq %rax, %r12
67c67
<    movq %rax, -24(%rbp)
---
>    movq %rax, %r13
70c70
<    movq %rax, -32(%rbp)
---
>    movq %rax, %r14
79c79

```

```

<    movq  -24(%rbp), %rdi
---
>    movq  %r13, %rdi
96c96
<    movq  -32(%rbp), %rdi
---
>    movq  %r14, %rdi
107c107
<    movq  %rax, -16(%rbp)
---
>    movq  %rax, %r13
116c116
<    movq  -8(%rbp), %rdi
---
>    movq  %r15, %rdi
133c133
<    movq  -16(%rbp), %rdi
---
>    movq  %r13, %rdi
144c144
<    movq  %rax, -48(%rbp)
---
>    movq  %rax, %r13
153c153
<    movq  -40(%rbp), %rdi
---
>    movq  %r12, %rdi
170c170
<    movq  -48(%rbp), %rdi
---
>    movq  %r13, %rdi

```

Lines from before the optimization are marked with a left arrow, and lines from after the optimization are marked with a right arrow. Without register allocation, our compiler assigns variables *a* through *f* to the stack, at offsets -8, -16, -24, -32, -40, and -48, respectively. This diff highlights how after our register allocation is done, our compiler produces assembly code that accesses these variables through a register assigned to them. Variable *a* is accessed from `%r15`, *b* is accessed from `%r13`, *c* is accessed from `%r13`, *d* is accessed from `%r14`, *e* is accessed from `%r12`, and *f* is

accessed from %r13. This is a sensible coloring, since b , c , and f do not interfere with each other in any way. Since these variables are accessed from actual registers instead of from the stack, this should result in a significant performance improvement as accessing registers is much faster than memory.

Our register allocator optimization was benchmarked against the derby test, and compared against no optimization, and with all dataflow optimizations and algebraic simplification enabled as well. Without any optimizations, the average run time of the derby test on Athena is 804 ms. With only register allocation enabled, the average run time of the derby test decreases to 699 ms. When combined with all dataflow optimizations and arithmetic simplification, the average run time of the derby test is reduced to 553 ms. This means that with all of our optimizations enabled, programs should run about 45% faster than without any optimization.

Loop invariant code detection

We did consider implementing loop invariant code detection. Our high-level approach to this would be to determine what instructions in a loop body actually depend on the iteration variables that are in scope. We would then use a dominator tree to take the instructions which do not depend on the iteration variable, and move them into the highest possible scope depending on the dominator tree. Upon analysis of the derby code, however, we found that many loops within the program do not have any loop invariant code. As a result, we decided to not implement loop invariant code detection, and instead focused our efforts on debugging and improving performance of our register allocator and dataflow optimizations.

Full Optimizations

To run all optimizations, you can pass `--opt=all` when running our compiler. This will run the following optimizations in this order:

1. Algebraic simplification
2. Common subexpression elimination
3. Copy propagation
4. Dead code elimination
5. Register allocation

Since register allocation does not change under multiple runs, it is run only once during code generation. Algebraic simplification, CSE, CP, and DCE may reveal further optimizations that can be performed by each other, so they are run in a loop up to 5 times. For example, combining Algebraic simplification (of constants) with CP can enable constant propagation. Regarding the number of iterations, we found that running the optimizations 5 times provides the best balance between compiler run time and

number of instructions removed, with approximately 60 lines being removed from the program provided for the derby when compared to only running the optimizations once.

Before register allocation completes, algebraic simplification is run first since it removes many constants that dataflow optimizations would redundantly operate on. Then CSE is run, removing common subexpressions and generating many temporary variables, which may be removed by further optimizations. After CSE completes, CP is run, which takes the temporary variable values and propagates them through the program. Since these propagations may end up producing dead code, then dead code elimination is run to remove unnecessary propagations and removes unused lines. Once the control flow graph has been optimized, then register allocation can be performed.

Contribution

For our dataflow optimizations, Matt implemented CSE at the block level, and Noah expanded upon this implementation to enable cross-block CSE. Both Noah and Matt debugged the implementation after expansion, with some insight provided by Julius and Adriano. Adriano implemented an interface for doing lattice-based dataflow optimizations. Subsequently, Julius implemented CP and Adriano implemented DCE using this lattice interface.

For further optimizations, Noah implemented algebraic simplification, which both Adriano and Noah subsequently further fleshed out to do constant computations and strength reduction, respectively. Adriano implemented def-use chains, the data structures for dealing with creating and interacting with webs (such as determining if webs intersect), and a generic graph interface for graph coloring. Matt implemented the greedy graph coloring algorithm, retrieving the spill cost for a web, modifying the interference graph based on node degree, and the logic required for assigning colors to webs. Both Noah and Julius refactored our code generation to support using our graph coloring allocator to assign variables to registers optimally. After our graph coloring register allocator was implemented, Noah, Julius, and Adriano were responsible for debugging the register allocator implementation. Finally, Matt was responsible for this writeup.